

DSA0604 Data Handling and Visualization - Slot B

Unit I - Class Activity 2

15/07/2026

1) Scenario:

Smart City Air Quality Monitoring: A city has installed 150 air quality sensors across different locations. The sensors record PM2.5, NO2, CO, temperature, and humidity every hour. Decision-makers struggle to identify pollution hotspots because of overlapping values.

Python:

```
# Smart City Air Quality Monitoring Visualization
```

```
import pandas as pd
```

```
import numpy as np
```

```
import matplotlib.pyplot as plt
```

```
# Generate Sample Data
```

```
np.random.seed(42)
```

```
n = 150
```

```
data = pd.DataFrame({  
    'Location': np.arange(1, n+1),  
    'PM25': np.random.randint(20, 180, n),  
    'NO2': np.random.randint(10, 120, n),  
    'CO': np.random.uniform(0.2, 5.0, n),  
    'Temperature': np.random.randint(20, 40, n),  
    'Humidity': np.random.randint(30, 90, n),  
    'X': np.random.uniform(0, 100, n),  
    'Y': np.random.uniform(0, 100, n)
```

```
})
```

```
# -----
```

```
# Sequential Color Palette
```

```
# -----
```

```
plt.figure(figsize=(7,6))
```

```
plt.scatter(data["X"], data["Y"],  
            c=data["PM25"],  
            cmap="Blues",  
            s=80)
```

```
plt.colorbar(label="PM2.5")
```

```
plt.title("Sequential Palette - PM2.5 Levels")
```

```
plt.xlabel("X Coordinate")
```

```
plt.ylabel("Y Coordinate")
```

```
plt.show()
```

```
# -----
```

```
# Diverging Color Palette
```

```
# -----
```

```
plt.figure(figsize=(7,6))
```

```
plt.scatter(data["X"], data["Y"],  
            c=data["Temperature"],  
            cmap="coolwarm",  
            s=80)
```

```
plt.colorbar(label="Temperature")
```

```
plt.title("Diverging Palette - Temperature")
```

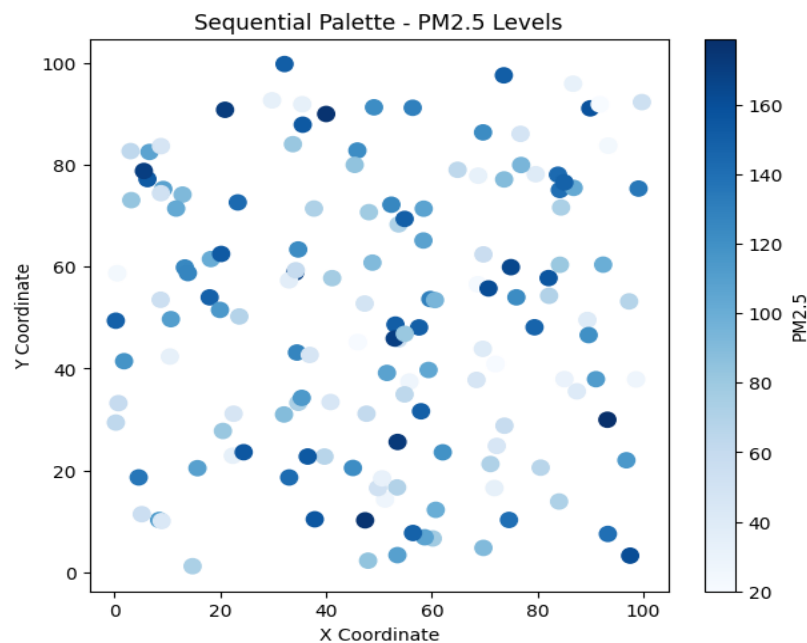
```
plt.xlabel("X Coordinate")
```

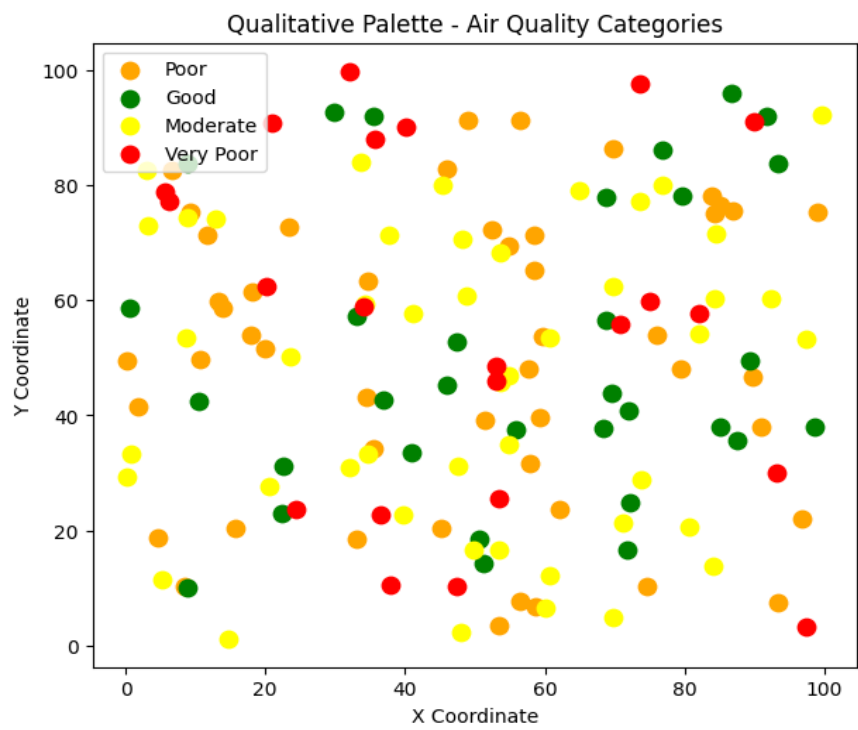
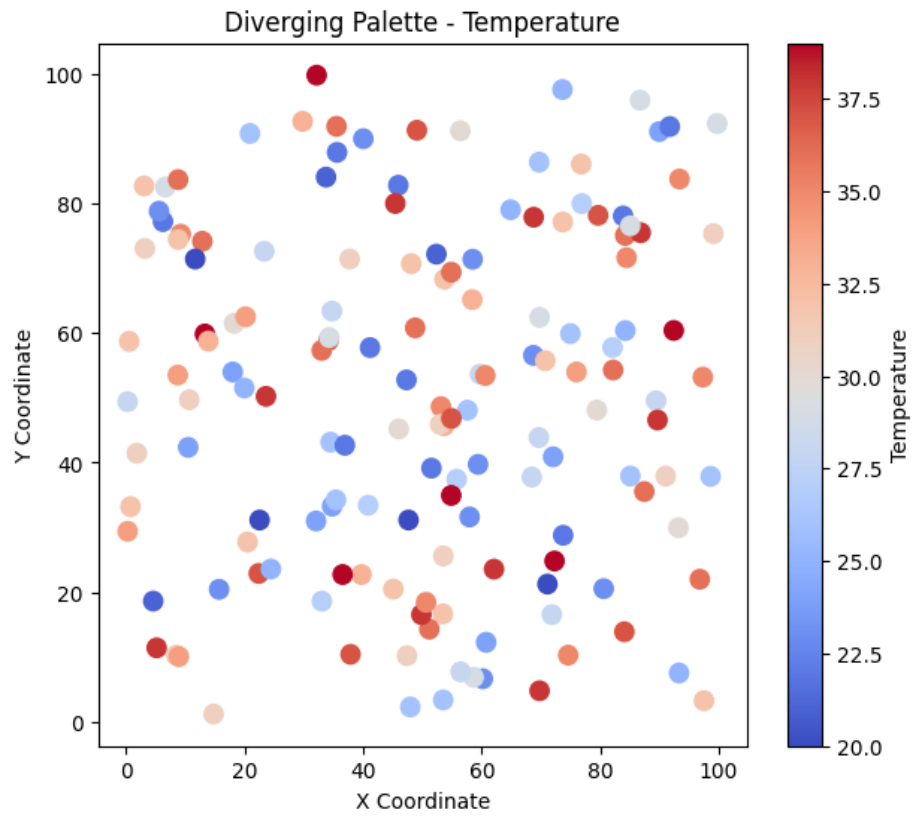
```

plt.ylabel("Y Coordinate")
plt.show()
# -----
# Qualitative Color Palette
# -----
category = pd.cut(data["PM25"],
                  bins=[0,50,100,150,200],
                  labels=["Good","Moderate","Poor","Very Poor"])

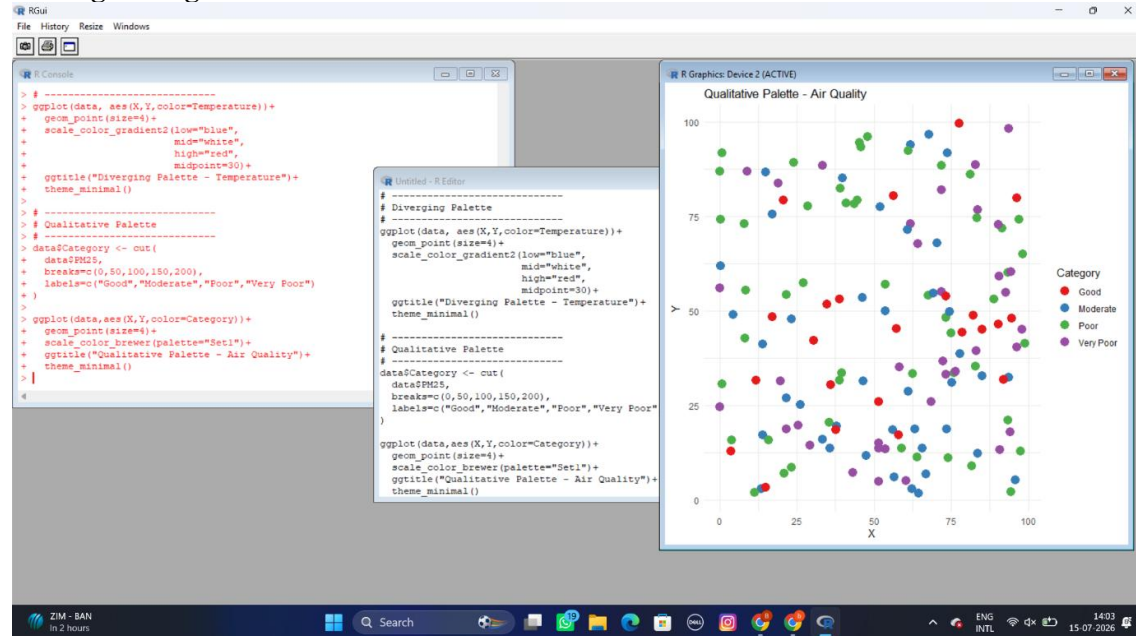
colors = {
    "Good":"green",
    "Moderate":"yellow",
    "Poor":"orange",
    "Very Poor":"red"
}
plt.figure(figsize=(7,6))
for c in category.unique():
    subset = data[category==c]
    plt.scatter(subset["X"],
                subset["Y"],
                color=colors[c],
                label=c,
                s=80)
plt.legend()
plt.title("Qualitative Palette - Air Quality Categories")
plt.xlabel("X Coordinate")
plt.ylabel("Y Coordinate")
plt.show()

```





R-Programing-



2)Scenario:

Earthquake Risk Analysis: A geological survey contains earthquake magnitude (0.5–9.5), depth (1–700 km), latitude, longitude, and aftershock frequency. Small earthquakes dominate the dataset, making large events difficult to interpret.

Python:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
```

```
# Generate Sample Earthquake Data
np.random.seed(42)
```

```
n = 1000
```

```
data = pd.DataFrame({
    'Magnitude': np.random.exponential(scale=1.2, size=n) + 0.5,
    'Depth': np.random.randint(1, 701, n),
    'Latitude': np.random.uniform(-90, 90, n),
    'Longitude': np.random.uniform(-180, 180, n),
    'Aftershocks': np.random.randint(0, 500, n)
})
```

```
# Limit magnitude between 0.5 and 9.5
data["Magnitude"] = data["Magnitude"].clip(0.5, 9.5)
```

```
# -----
# Linear Scale Scatter Plot
# -----
plt.figure(figsize=(8,6))
```

```
plt.scatter(data["Magnitude"],
            data["Aftershocks"],
            c=data["Depth"],
            cmap="viridis",
            alpha=0.7)

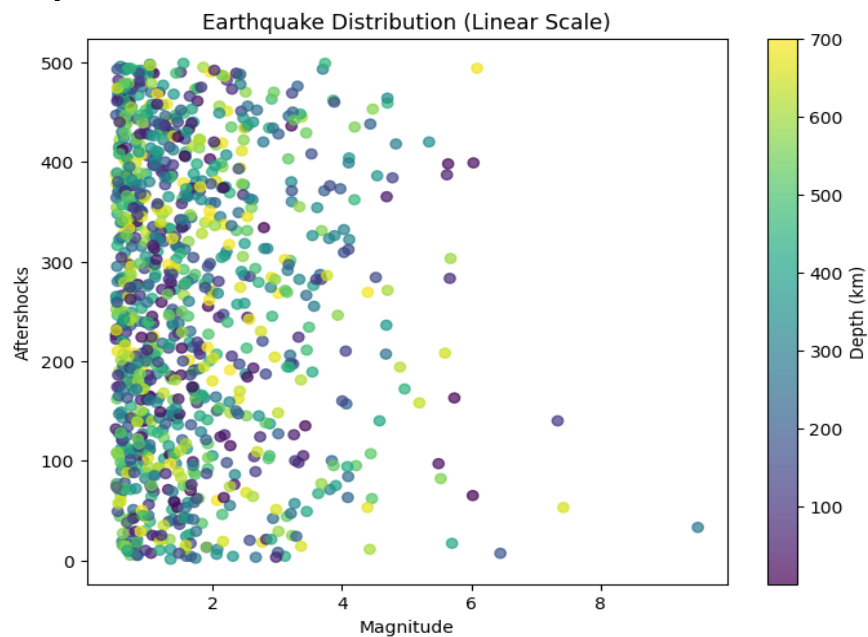
plt.colorbar(label="Depth (km)")
plt.title("Earthquake Distribution (Linear Scale)")
plt.xlabel("Magnitude")
plt.ylabel("Aftershocks")
plt.show()
```

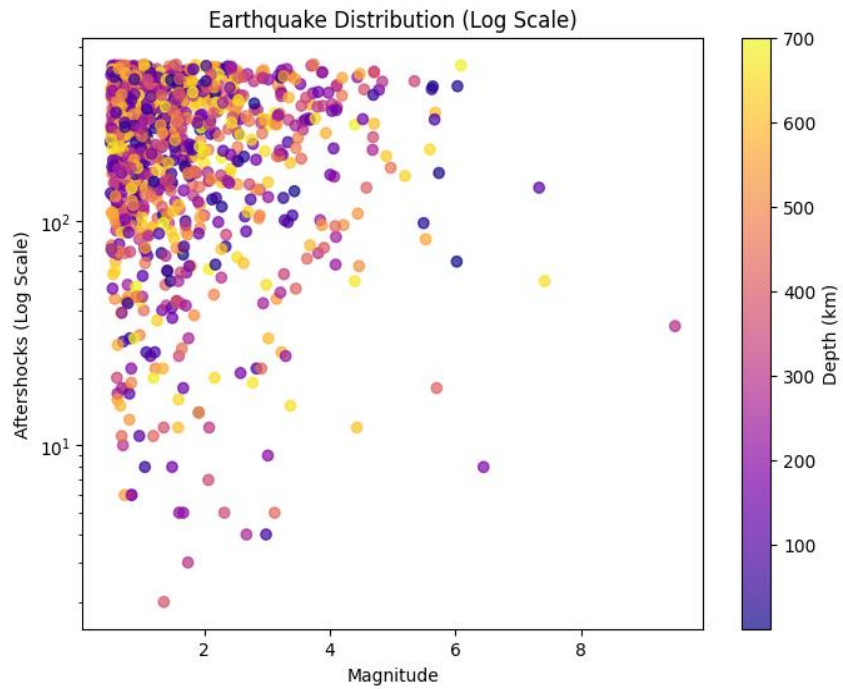
```
# -----
# Logarithmic Scale Scatter Plot
# -----
```

```
plt.figure(figsize=(8,6))
plt.scatter(data["Magnitude"],
            data["Aftershocks"]+1,
            c=data["Depth"],
            cmap="plasma",
            alpha=0.7)

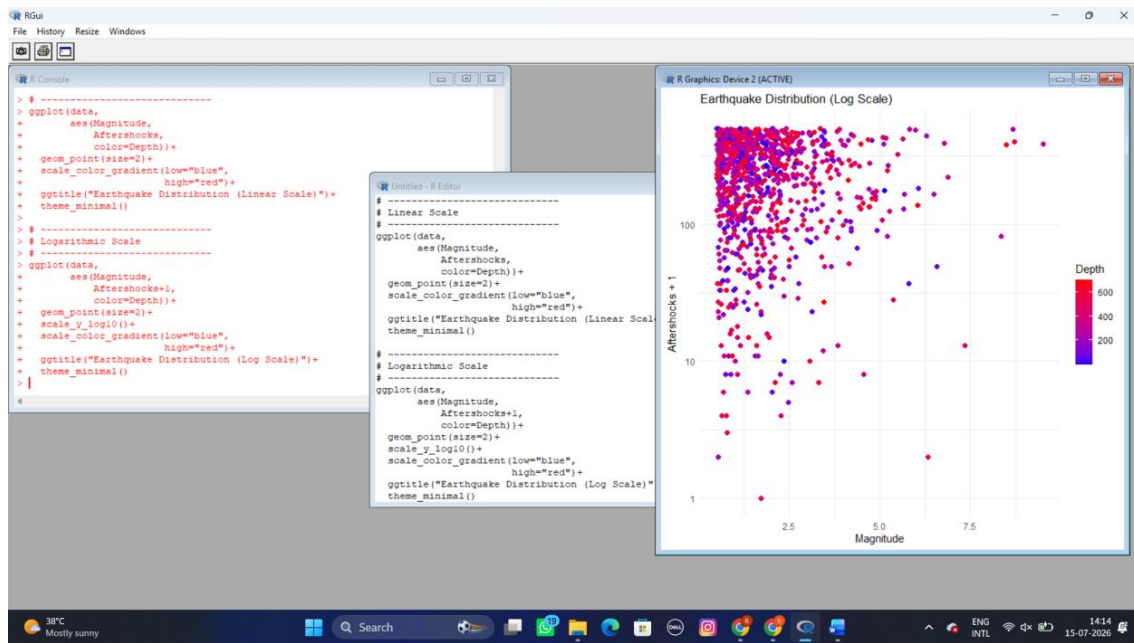
plt.yscale("log")
plt.colorbar(label="Depth (km)")
plt.title("Earthquake Distribution (Log Scale)")
plt.xlabel("Magnitude")
plt.ylabel("Aftershocks (Log Scale)")
plt.show()
```

Output:





R-Programming:



3)Scenario:

Wind Direction Analysis for Airport Operations: An airport collects hourly wind speed and wind direction for one year. Pilots need an intuitive visualization showing prevailing wind directions and seasonal variations.

Python:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

# Generate Sample Data
np.random.seed(42)

n = 8760 # Hourly data for one year

wind_speed = np.random.uniform(2, 25, n)
wind_direction = np.random.uniform(0, 360, n)

data = pd.DataFrame({
    "WindSpeed": wind_speed,
    "WindDirection": wind_direction
})

# -----
# Polar Plot (Wind Rose)
# -----
fig = plt.figure(figsize=(7,7))
ax = fig.add_subplot(111, polar=True)

theta = np.deg2rad(data["WindDirection"])
r = data["WindSpeed"]

ax.scatter(theta, r, alpha=0.5)

ax.set_theta_zero_location("N")
ax.set_theta_direction(-1)

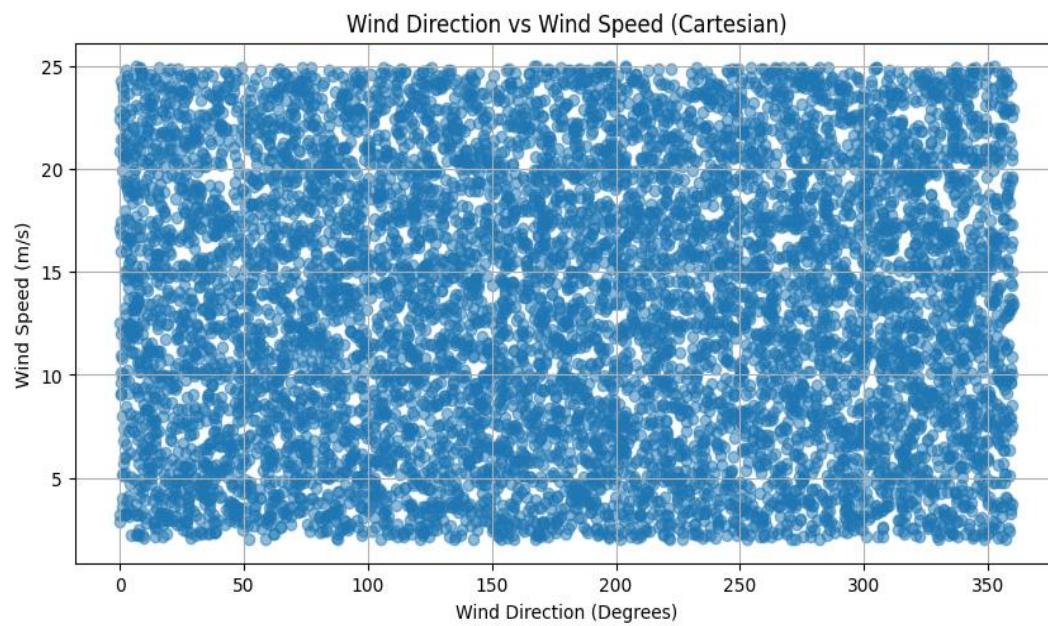
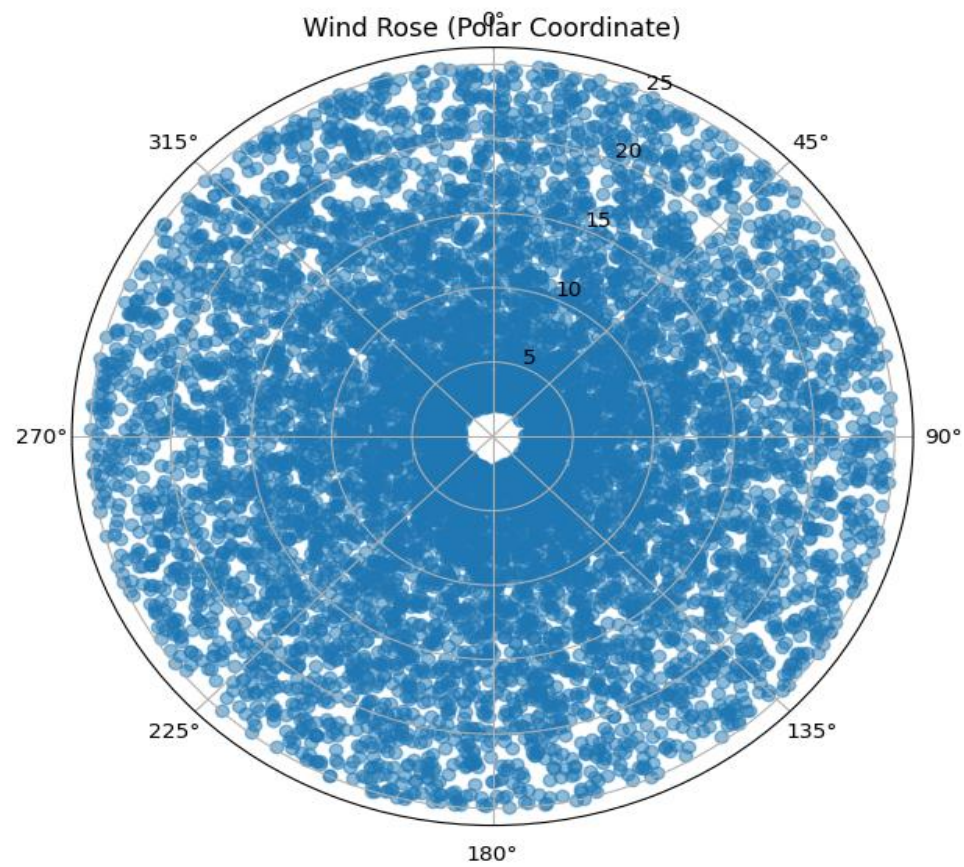
plt.title("Wind Rose (Polar Coordinate)")
plt.show()

# -----
# Cartesian Plot
# -----
plt.figure(figsize=(10,5))
plt.scatter(data["WindDirection"],
            data["WindSpeed"],
            alpha=0.5)

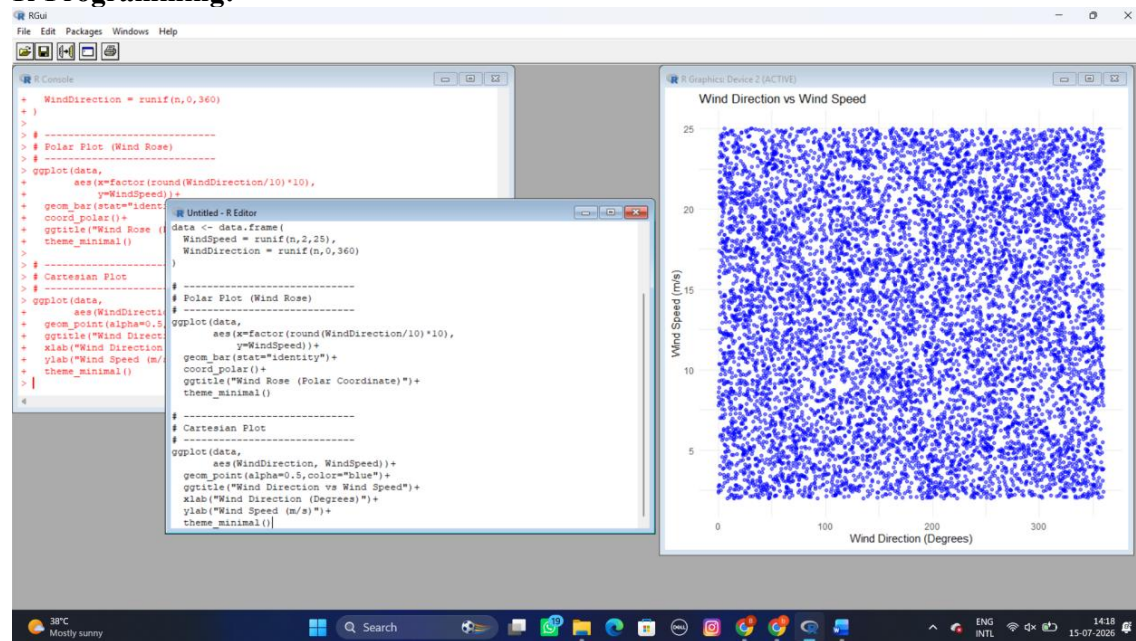
plt.xlabel("Wind Direction (Degrees)")
plt.ylabel("Wind Speed (m/s)")
plt.title("Wind Direction vs Wind Speed (Cartesian)")
```



```
plt.grid(True)
plt.show()
Output:
```



R-Programming:



4)Scenario:

Hospital Patient Monitoring Dashboard: ICU patients have heart rate, oxygen saturation, blood pressure, respiration rate, and temperature recorded every minute. Doctors require a dashboard that instantly highlights abnormal conditions.

Python:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
```

```
# Sample ICU Patient Data
np.random.seed(42)
```

```
patients = pd.DataFrame({
    "Patient": ["P1", "P2", "P3", "P4", "P5"],
    "HeartRate": np.random.randint(50,130,5),
    "Oxygen": np.random.randint(85,100,5),
    "BloodPressure": np.random.randint(80,170,5),
    "Respiration": np.random.randint(10,30,5),
    "Temperature": np.random.uniform(35,40,5).round(1)
})
```

```
# Function to assign colors
def get_color(value, low, high):
    if value < low or value > high:
        return "red"
    elif value < low+5 or value > high-5:
        return "yellow"
    else:
        return "green"
```

```

metrics = {
  "HeartRate": (60,100),
  "Oxygen": (95,100),
  "BloodPressure": (90,140),
  "Respiration": (12,20),
  "Temperature": (36.5,37.5)
}

# Dashboard
fig, axes = plt.subplots(1,5, figsize=(18,4))

for i, metric in enumerate(metrics):
  low, high = metrics[metric]
  colors = [get_color(v, low, high) for v in patients[metric]]

  axes[i].bar(patients["Patient"], patients[metric], color=colors)
  axes[i].set_title(metric)
  axes[i].tick_params(axis='x', rotation=45)

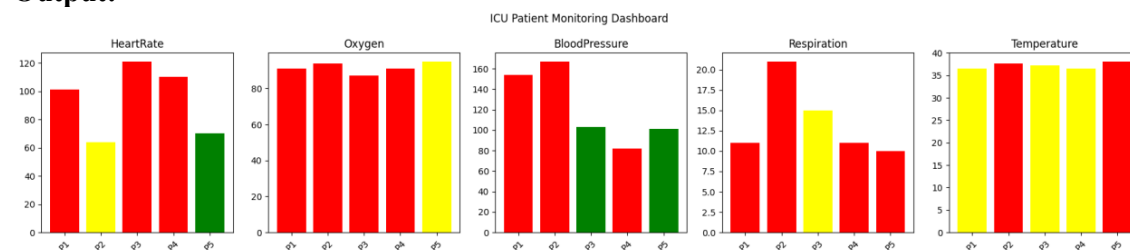
```

```

plt.suptitle("ICU Patient Monitoring Dashboard")
plt.tight_layout()
plt.show()

```

Output:



R-Programming:



5)Scenario:

Global Climate Change Study: Researchers analyze temperature anomalies across 180 countries from 1980–2025. The objective is to show both positive and negative deviations from historical averages.

Python:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

# Generate Sample Data
np.random.seed(42)

countries = [f"Country{i}" for i in range(1, 181)]
years = list(range(1980, 2026))

# Temperature anomalies (-3°C to +3°C)
data = np.random.uniform(-3, 3, (180, len(years)))

df = pd.DataFrame(data, index=countries, columns=years)

# -----
# Heatmap 1 - Coolwarm Palette
# -----
plt.figure(figsize=(12,6))
plt.imshow(df, cmap="coolwarm", aspect="auto")
plt.colorbar(label="Temperature Anomaly (°C)")
plt.title("Heatmap - Coolwarm Palette")
plt.xlabel("Year")
plt.ylabel("Countries")
plt.show()

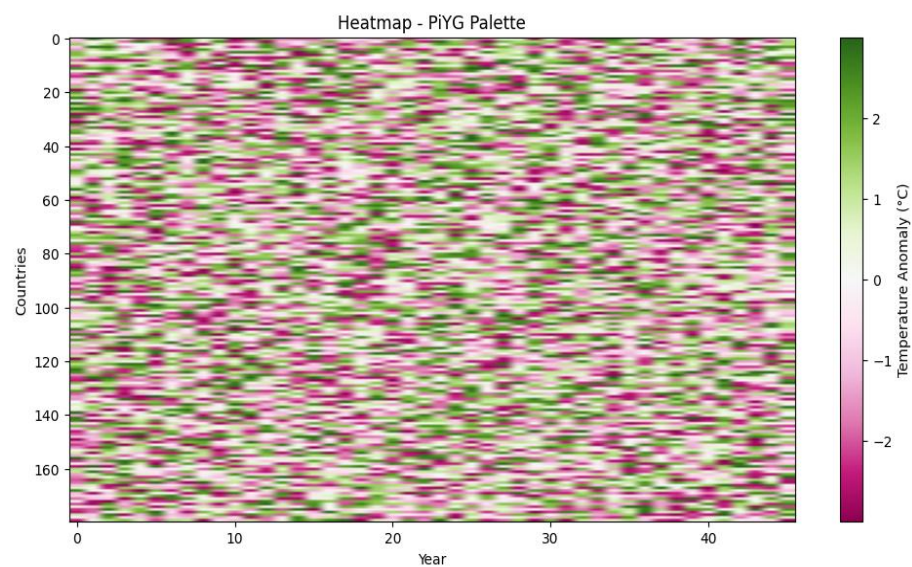
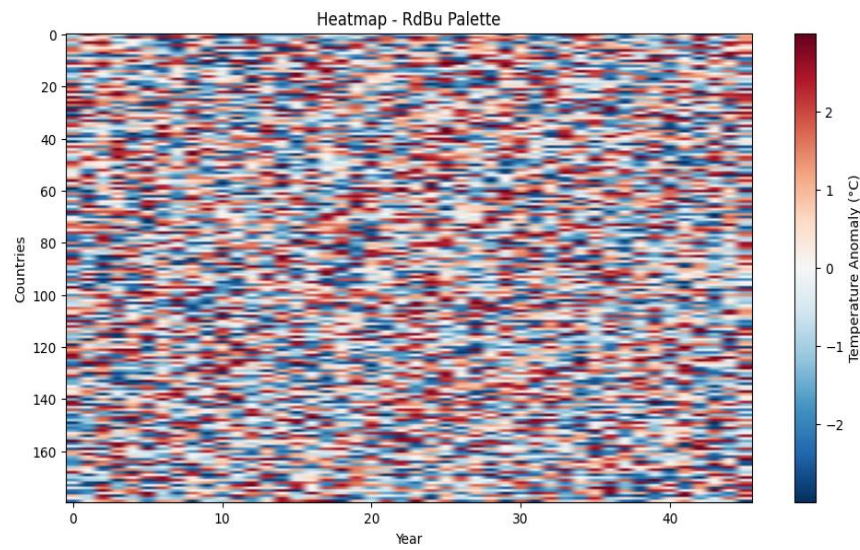
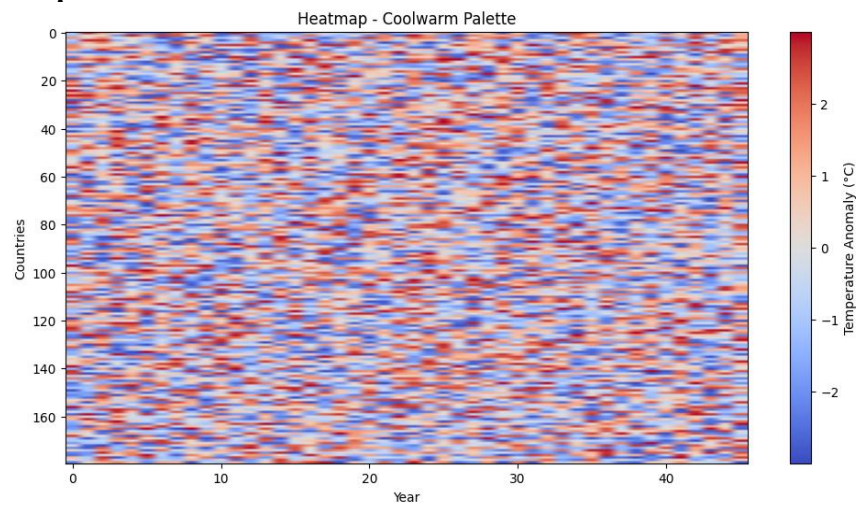
# -----
# Heatmap 2 - RdBu Palette
# -----
plt.figure(figsize=(12,6))
plt.imshow(df, cmap="RdBu_r", aspect="auto")
plt.colorbar(label="Temperature Anomaly (°C)")
plt.title("Heatmap - RdBu Palette")
plt.xlabel("Year")
plt.ylabel("Countries")
plt.show()

# -----
# Heatmap 3 - PiYG Palette
# -----
plt.figure(figsize=(12,6))
plt.imshow(df, cmap="PiYG", aspect="auto")
plt.colorbar(label="Temperature Anomaly (°C)")
plt.title("Heatmap - PiYG Palette")
plt.xlabel("Year")
```

```
plt.ylabel("Countries")
```

```
plt.show()
```

Output:



6)Scenario:

E-Commerce Sales Analytics: A retail company sells over 50 product categories across 30 regions. Management wants to distinguish categories while simultaneously representing sales volume.

Python:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

# Generate Sample Data
np.random.seed(42)

n = 300

categories = ['Electronics', 'Clothing', 'Furniture', 'Sports', 'Food']
regions = ['North', 'South', 'East', 'West', 'Central']

data = pd.DataFrame({
    'Category': np.random.choice(categories, n),
    'Region': np.random.choice(regions, n),
    'Sales': np.random.randint(1000, 10000, n)
})

# Assign colors for categories
category_colors = {
    'Electronics': 'red',
    'Clothing': 'blue',
    'Furniture': 'green',
    'Sports': 'orange',
    'Food': 'purple'
}

# Scatter Plot
plt.figure(figsize=(10,6))

for cat in categories:
    subset = data[data['Category']==cat]
    plt.scatter(subset['Region'],
                subset['Sales'],
                s=subset['Sales']/50,      # Bubble size = Sales
                color=category_colors[cat], # Category Color
                alpha=0.6,
                label=cat)

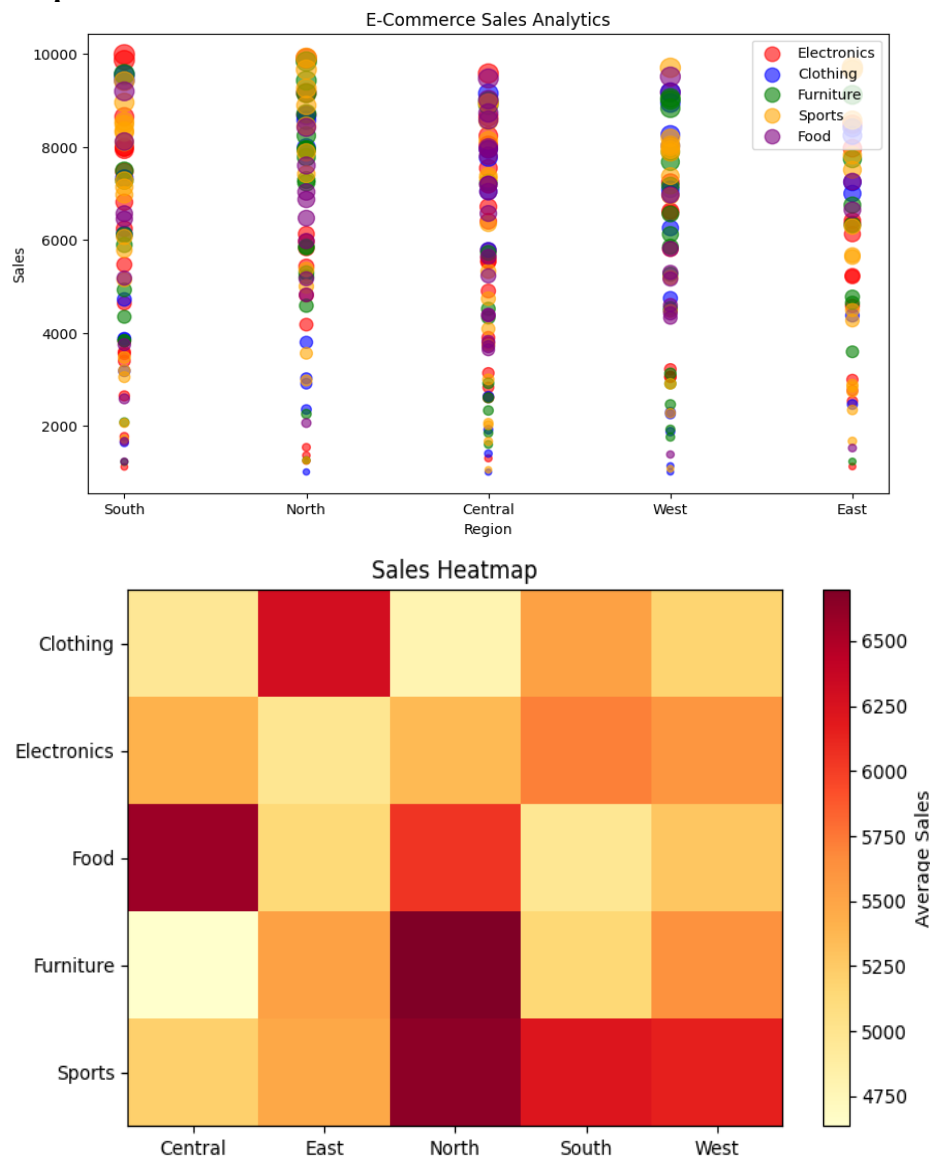
plt.xlabel("Region")
plt.ylabel("Sales")
plt.title("E-Commerce Sales Analytics")
plt.legend()
plt.show()
```

```
# Heatmap for Sales Volume
pivot = data.pivot_table(values='Sales',
                           index='Category',
                           columns='Region',
                           aggfunc='mean')

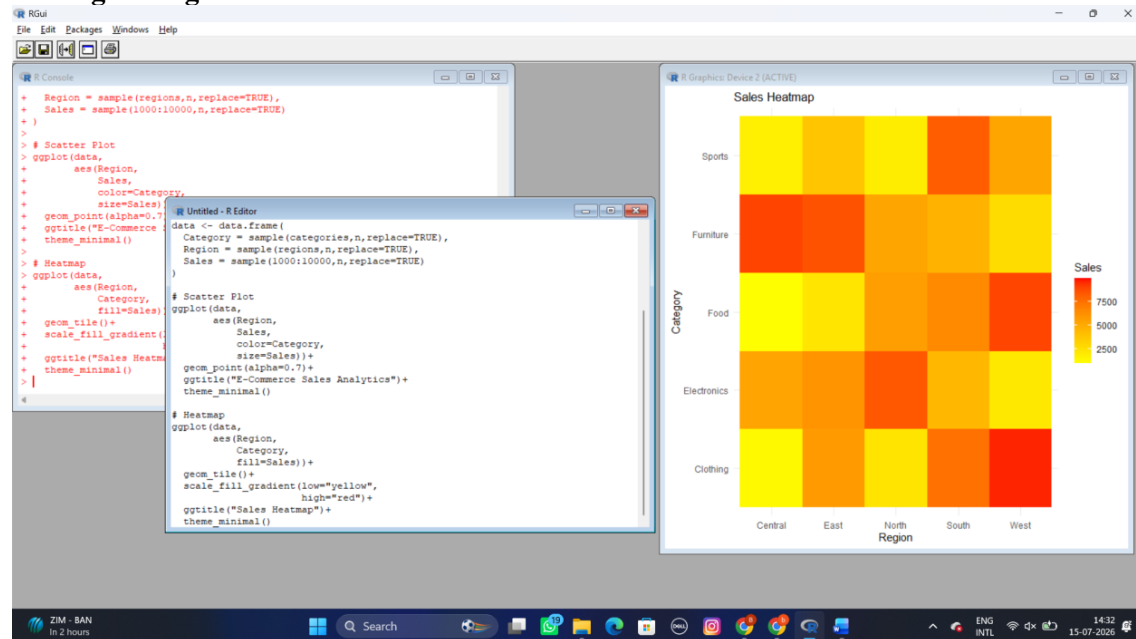
plt.figure(figsize=(8,5))
plt.imshow(pivot, cmap='YlOrRd', aspect='auto')

plt.xticks(range(len(pivot.columns)), pivot.columns)
plt.yticks(range(len(pivot.index)), pivot.index)
plt.colorbar(label="Average Sales")
plt.title("Sales Heatmap")
plt.show()
```

Output:



R-Programming:



7)Scenario:

Satellite Image Vegetation Analysis: A satellite dataset provides NDVI values ranging from -1 to $+1$. Environmental scientists must identify healthy vegetation, barren land, and water bodies.

Python:

```
import numpy as np
```

```
import matplotlib.pyplot as plt
```

```
# Generate Sample NDVI Data (-1 to +1)
```

```
np.random.seed(42)
```

```
ndvi = np.random.uniform(-1, 1, (100, 100))
```

```
# -----
```

```
# Sequential Palette (Greens)
```

```
# -----
```

```
plt.figure(figsize=(6,5))
```

```
plt.imshow(ndvi, cmap="Greens", vmin=-1, vmax=1)
```

```
plt.colorbar(label="NDVI")
```

```
plt.title("Sequential Palette - Greens")
```

```
plt.axis("off")
```

```
plt.show()
```

```
# -----
```

```
# Sequential Palette (YlGn)
```

```
# -----
```

```
plt.figure(figsize=(6,5))
```

```
plt.imshow(ndvi, cmap="YlGn", vmin=-1, vmax=1)
```

```
plt.colorbar(label="NDVI")
```

```
plt.title("Sequential Palette - Yellow-Green")
```



```

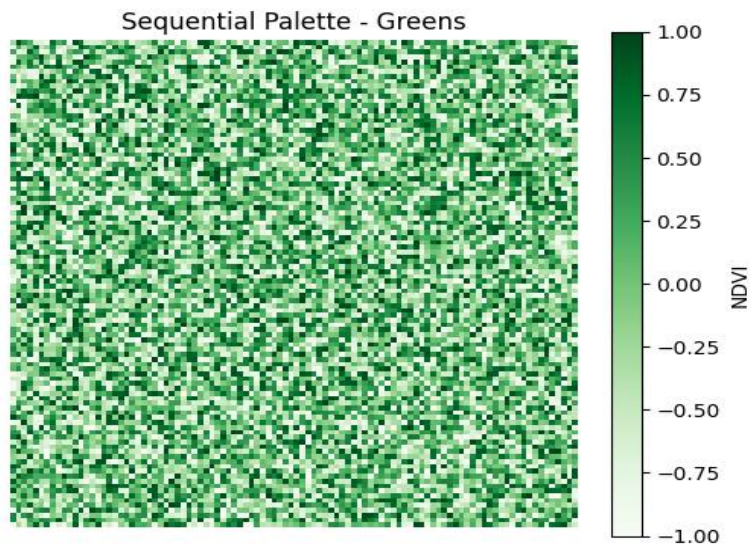
plt.axis("off")
plt.show()

# -----
# Diverging Palette (RdYlGn)
# -----
plt.figure(figsize=(6,5))
plt.imshow(ndvi, cmap="RdYlGn", vmin=-1, vmax=1)
plt.colorbar(label="NDVI")
plt.title("Diverging Palette - Red Yellow Green")
plt.axis("off")
plt.show()

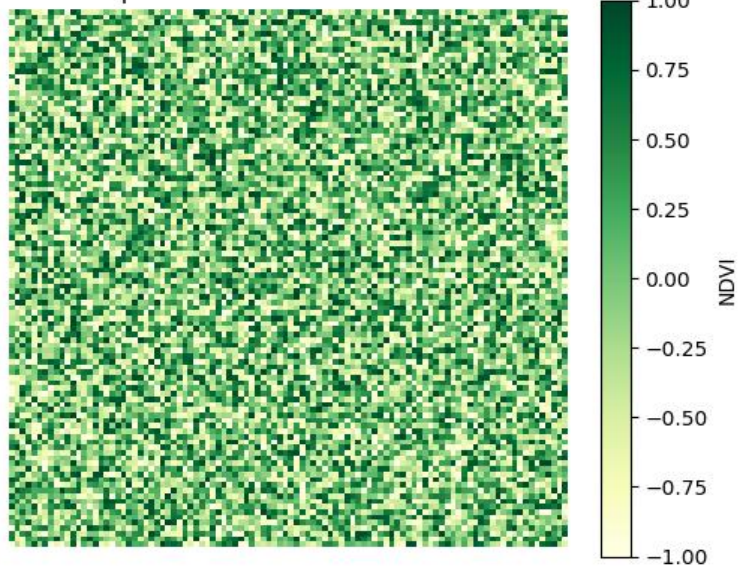
# -----
# Diverging Palette (BrBG)
# -----
plt.figure(figsize=(6,5))
plt.imshow(ndvi, cmap="BrBG", vmin=-1, vmax=1)
plt.colorbar(label="NDVI")
plt.title("Diverging Palette - Brown Blue Green")
plt.axis("off")
plt.show()

```

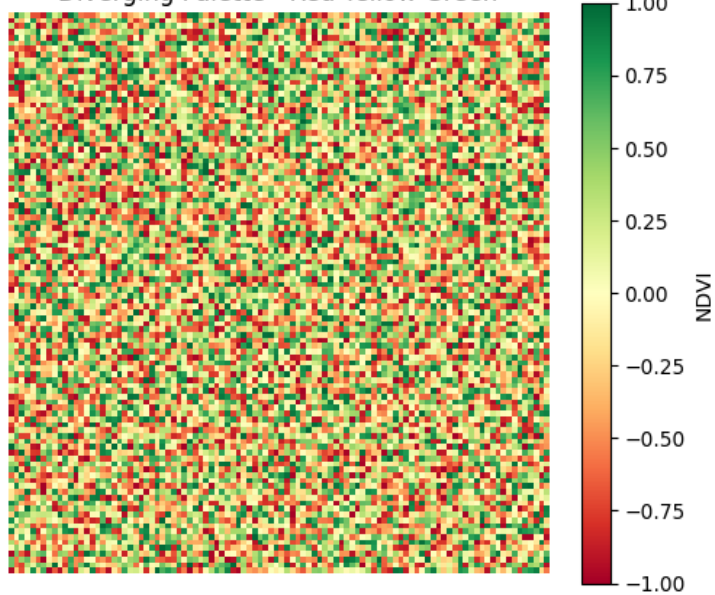
Output:



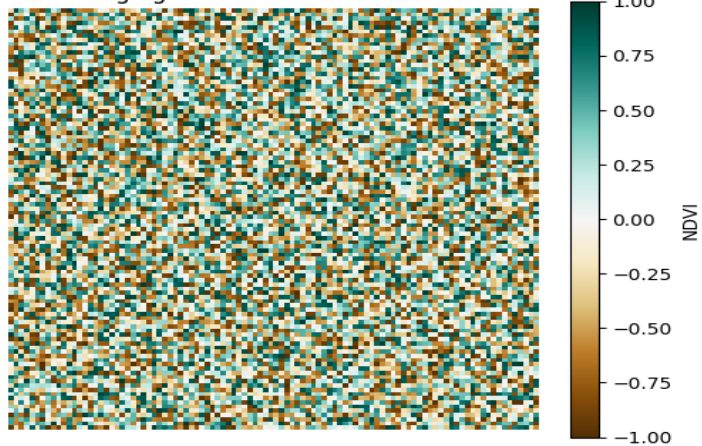
Sequential Palette - Yellow-Green



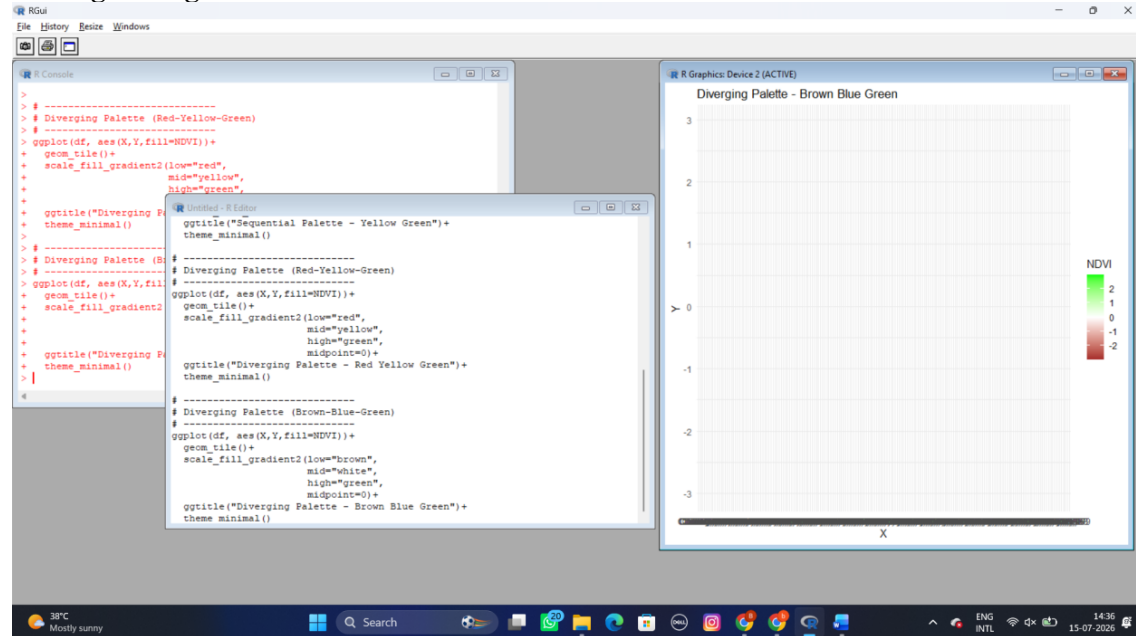
Diverging Palette - Red Yellow Green



Diverging Palette - Brown Blue Green



R-Programming:



8)Scenario:

Cryptocurrency Market Volatility: Daily prices of Bitcoin, Ethereum, Solana, and Cardano span several years. Price values differ by several orders of magnitude, making direct comparison difficult.

Python:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
```

```
# Generate Sample Data
np.random.seed(42)
```

```
days = np.arange(1, 366)
```

```
bitcoin = np.random.lognormal(mean=10.5, sigma=0.15, size=365)
ethereum = np.random.lognormal(mean=8.5, sigma=0.18, size=365)
solana = np.random.lognormal(mean=5.8, sigma=0.25, size=365)
cardano = np.random.lognormal(mean=4.8, sigma=0.30, size=365)
```

```
# -----
# Cartesian Plot
# -----
plt.figure(figsize=(10,5))
```

```
plt.plot(days, bitcoin, color='orange', label='Bitcoin')
plt.plot(days, ethereum, color='blue', label='Ethereum')
plt.plot(days, solana, color='green', label='Solana')
plt.plot(days, cardano, color='red', label='Cardano')
```

```
plt.xlabel("Days")
```

```

plt.ylabel("Price (USD)")
plt.title("Cryptocurrency Prices (Cartesian Scale)")
plt.legend()
plt.grid(True)
plt.show()

# -----
# Logarithmic Scale Plot
# -----
plt.figure(figsize=(10,5))

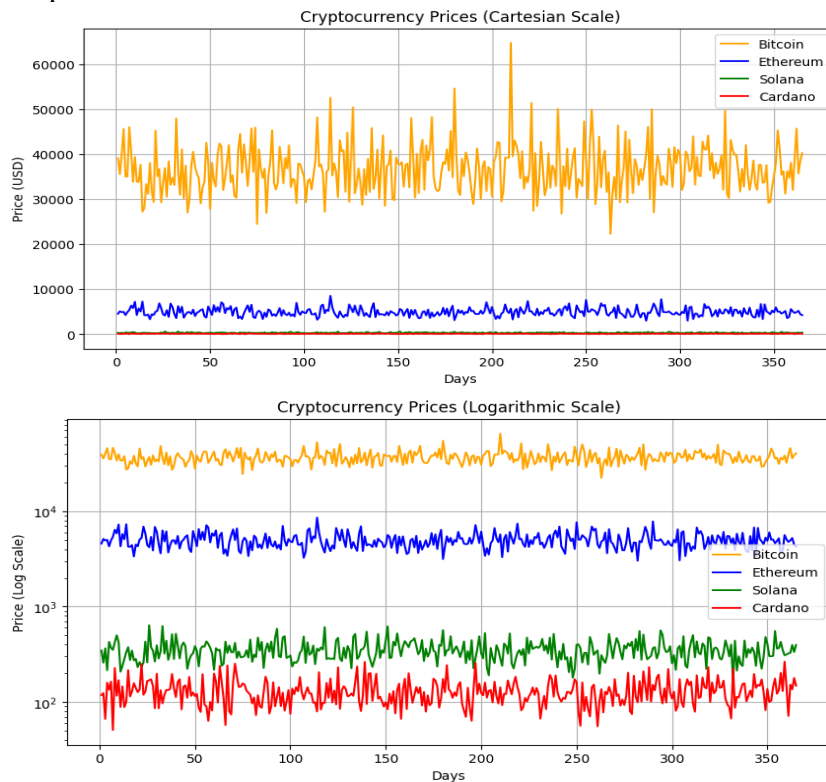
plt.plot(days, bitcoin, color='orange', label='Bitcoin')
plt.plot(days, ethereum, color='blue', label='Ethereum')
plt.plot(days, solana, color='green', label='Solana')
plt.plot(days, cardano, color='red', label='Cardano')

plt.yscale('log')

plt.xlabel("Days")
plt.ylabel("Price (Log Scale)")
plt.title("Cryptocurrency Prices (Logarithmic Scale)")
plt.legend()
plt.grid(True)
plt.show()

```

Output:



The screenshot displays the R Studio environment. The top-left pane shows the R Console with the following code:

```

+ "Solana", "Cardano"),
+   each=365)
+ )
+
+ # -----
+ > # Cartesian Plot
+ # -----
+ > ggplot(data,
+   aes(Day, Price,
+   color=Crypto))
+   geom_line()
+   ggtitle("Cryptocurrency
+   theme_minimal()
+ )
+ # -----
+ > # Logarithmic Plot
+ # -----
+ > ggplot(data,
+   aes(Day, Price,
+   color=Crypto))
+   geom_line()
+   scale_y_log10()
+   ggtitle("Cryptocurrency
+   theme_minimal()
+ )

```

The bottom-left pane shows the R Graphics Device 2 (ACTIVE) window, titled "Cryptocurrency Prices (Logarithmic Scale)". It displays a line plot with four series: Bitcoin (red), Cardano (teal), Ethereum (purple), and Solana (green). The y-axis is labeled "Price" and ranges from 100 to 10,000 on a logarithmic scale. The x-axis is labeled "Day" and ranges from 0 to 300. The plot shows the price fluctuations of these cryptocurrencies over time.

Marathon Runner Performance Analysis: A marathon organizer records runner speed, heart rate, distance covered, elevation, and running direction throughout the race. Coaches want to analyze pacing strategies and directional movement.

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
```

```
distance = np.linspace(0, 42.2, n)          # Distance (km)
speed = np.random.uniform(8, 18, n)         # Speed (km/h)
heart_rate = np.random.randint(110, 190, n)
elevation = np.random.randint(0, 300, n)
direction = np.random.uniform(0, 360, n)    # Direction (degrees)
```

```
scatter = plt.scatter(distance,  
                      speed,  
                      c=speed,  
                      cmap="viridis",  
                      s=40)
```

```

plt.colorbar(scatter, label="Speed (km/h)")
plt.xlabel("Distance Covered (km)")
plt.ylabel("Speed (km/h)")
plt.title("Runner Speed vs Distance (Cartesian)")
plt.grid(True)
plt.show()

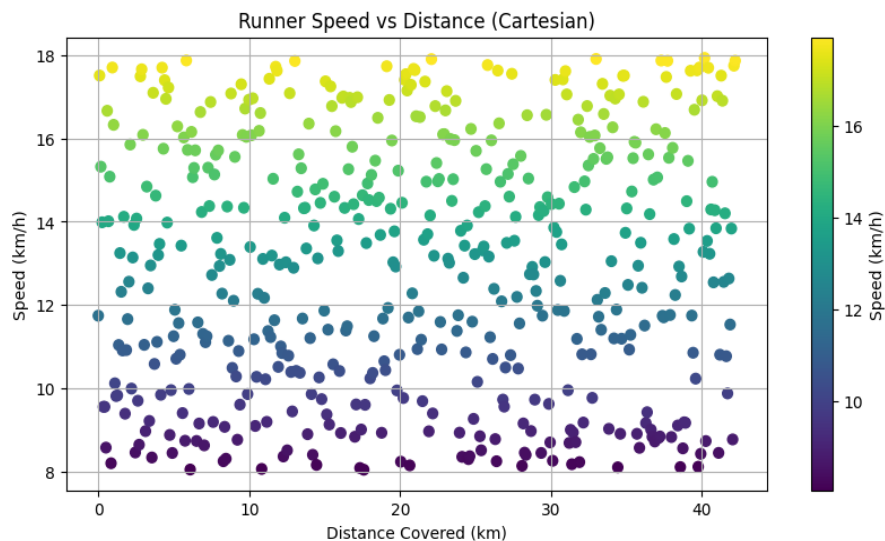
# -----
# Polar Coordinate Plot
# -----
plt.figure(figsize=(7,7))
ax = plt.subplot(111, polar=True)

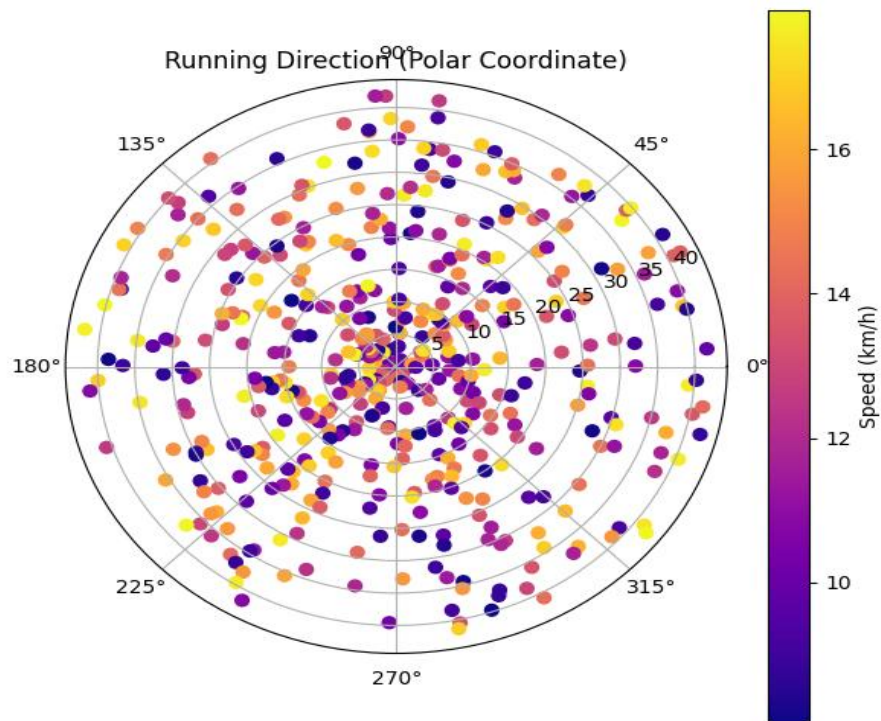
theta = np.deg2rad(direction)

scatter = ax.scatter(theta,
                      distance,
                      c=speed,
                      cmap="plasma",
                      s=40)

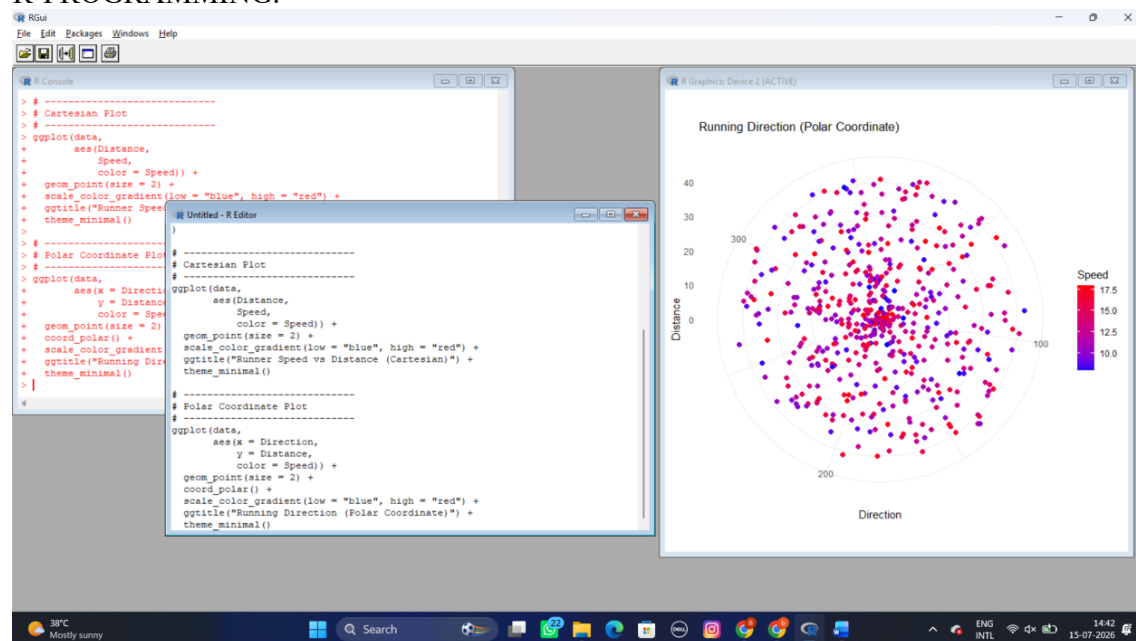
plt.colorbar(scatter, label="Speed (km/h)")
ax.set_title("Running Direction (Polar Coordinate)")
plt.show()
Output:

```





R-PROGRAMMING:



10)Scenario:

COVID-19 Vaccination Campaign Dashboard: Health officials monitor vaccination rates across districts, age groups, and vaccine types. They need visualizations suitable for both policymakers and the general public.

Python:

```
import numpy as np
import pandas as pd
```



```

import matplotlib.pyplot as plt

# Generate Sample Data
np.random.seed(42)

districts = [f"D{i}" for i in range(1,11)]
age_groups = ["0-17", "18-44", "45-60", "60+"]
vaccines = ["Covishield", "Covaxin", "Sputnik"]

data = pd.DataFrame({
    "District": np.random.choice(districts,100),
    "AgeGroup": np.random.choice(age_groups,100),
    "Vaccine": np.random.choice(vaccines,100),
    "VaccinationRate": np.random.randint(40,101,100)
})

# -----
# Version 1: Policymakers Dashboard
# -----
pivot = data.pivot_table(values="VaccinationRate",
                           index="District",
                           columns="AgeGroup",
                           aggfunc="mean")

plt.figure(figsize=(8,6))
plt.imshow(pivot, cmap="YlGn", aspect="auto")
plt.colorbar(label="Vaccination Rate (%)")
plt.xticks(range(len(pivot.columns)), pivot.columns)
plt.yticks(range(len(pivot.index)), pivot.index)
plt.title("Vaccination Dashboard (Policymakers)")
plt.show()

# -----
# Version 2: General Public Dashboard
# -----
colors = {
    "Covishield": "blue",
    "Covaxin": "orange",
    "Sputnik": "green"
}

plt.figure(figsize=(8,5))

for vaccine in vaccines:
    subset = data[data["Vaccine"]==vaccine]
    plt.scatter(subset["District"],
                subset["VaccinationRate"],
                color=colors[vaccine],
                label=vaccine,
                s=70)

```

```
plt.xlabel("District")
plt.ylabel("Vaccination Rate (%)")
plt.title("Vaccination Dashboard (General Public)")
plt.legend()
plt.grid(True)
plt.show()
```

Output:

